

GearGuard: A Detector-Agnostic Mobile Architecture for Personal Protective Equipment Compliance Monitoring

Ahmad Fauzan Prayogi (sole author)
github.com/afprayogi

Abstract—Verifying that workers wear the required personal protective equipment (PPE) before entering a work area is a routine but error-prone safety task. Existing research concentrates on the detection model, while the surrounding workflow—identifying the worker, applying site-specific rules, recording the result and producing audit evidence—receives little attention. This paper presents GearGuard, a cross-platform mobile application covering that workflow end to end. We contribute (i) a formal, detector-independent definition of per-scan and per-worker compliance, (ii) a layered architecture in which a single interface isolates the machine-learning model from the user interface, storage and reporting, and (iii) an open-source Flutter reference implementation with scan, attendance, ranking, archive and PDF-report features, verified by 38 automated tests (75% line coverage, 20 accessibility checks). A smoke test of the author's YOLO model detected the expected equipment class in 11 of 11 validation photos (about 80 ms per frame on a desktop CPU), while robustness and negative-image experiments expose weaknesses to occlusion (5/11 with a 25% patch) and to high-visibility colours (false *Vest* on plain orange). A usability study remains future work.

Keywords—personal protective equipment, occupational safety, mobile application, object detection, software architecture, Flutter

I. INTRODUCTION

Construction, manufacturing and logistics sites require workers to wear helmets, high-visibility vests, gloves and safety footwear. Compliance is usually checked by a supervisor at the gate, which is slow, inconsistent and leaves no structured record. Vision-based PPE detection has been studied extensively [1], [2], [5], building on real-time single-stage detectors such as the YOLO family [3], [4]. A detector alone, however, is not a usable safety tool: supervisors must also know *who* was scanned, *which* rules applied, and be able to retrieve history for audits.

Contributions: (1) a detector-independent formalisation of compliance; (2) a layered architecture treating the detector as a replaceable component; (3) an open-source implementation and its verification.

Prior work reports detection accuracy on site imagery [1], [2], [5]; the surrounding workflow (worker identity, site rules, records, audit reports) is rarely the object of study. This paper addresses that gap with a reference implementation that keeps the detector replaceable.

II. RELATED WORK

PPE detection. Wang et al. [1] released the CHV dataset (1,330 construction-site images) and compared eight YOLO-based detectors for helmets of four colours, persons and vests. Karlsson et al. [2] built a YOLOv4 system for five equipment categories (hard hats, vests, gloves, glasses, hearing protection) at entry-control points, the scenario closest to ours. Ordick et al. [5] trained YOLOv11s on 589 images (1,425 instances, seven categories) for electric-utility workers and reported 94.0% precision, 90.1% recall and 92.8% mAP@50, with a web application that processes an image in 2–3 s on a CPU; they note a small validation set and difficulty detecting safety footwear.

Detector families. Single-stage detectors such as YOLOv7 [3] and YOLOv8 [4] trade accuracy for latency and are the usual choice for on-device inference.

Gap. These works optimise or evaluate the model. They do not specify the application layer that turns detections into decisions and records: the compliance rules, per-worker aggregation, attendance and audit output. GearGuard formalises that layer (Section III) and isolates the model behind one interface (Section IV-B).

III. PROBLEM FORMULATION

Let P be the set of supported PPE types (helmet, vest, gloves, boots, goggles, mask). A site fixes a required subset $R \subseteq P$, $R \neq \emptyset$, and a confidence threshold $\tau \in [0,1]$. For a frame, the detector returns a confidence $c_p \in [0,1]$ for every $p \in P$. An item is *detected* when $c_p \geq \tau$. The set of missing items is

$$M = \{p \in R \mid c_p < \tau\}. \quad (1)$$

A scan is *compliant* iff $M = \emptyset$, and its *score* is

$$s = (|R| - |M|) / |R|. \quad (2)$$

For a worker w with scan set S_w , the compliance rate is

$$C_w = |\{x \in S_w \mid M_x = \emptyset\}| / |S_w|, \quad (3)$$

and workers are ranked by C_w in descending order, ties broken by the mean score. The daily site rate is the same ratio over all scans of that day. R and τ are stored with each scan, so historical results stay valid when site rules change.

IV. SYSTEM DESIGN

A. Layered architecture

GearGuard has four layers: *domain* (pure Dart value types), *data* (detector, storage, PDF builder), *state* (flutter_bloc cubits with immutable states) and *presentation* (feature modules). Derived quantities— C_w , rankings, most-missed equipment—

are computed in the state layer, not in widgets, which keeps them unit-testable.

```
presentation  dashboard | scanner | attendance
              team | archive | settings
state         SafetyCubit | SettingsCubit
data          PpeDetector | SafetyRepository |
report_pdf
domain        PpeItem | Employee | ScanRecord
```

B. Detector interface

```
abstract interface class PpeDetector {
  Future<Map<PpeItem, double>> detect();
}
```

This is the only contract between the model and the rest of the app. It is injected at the application root, so replacing the simulated detector with an on-device model (e.g., a YOLO network through a platform channel) needs no change to any screen, cubit or report.

C. Interaction design

Bottom navigation exposes the five daily tasks (Home, Attendance, Scan, Team, Archive). Violations are sorted first in the attendance list; workers who have not checked in have a one-tap “Scan” action; the scan result shows a per-item confidence checklist with a bounding-box overlay (Fig. 1); status is never conveyed by colour alone. Light and dark themes are provided.

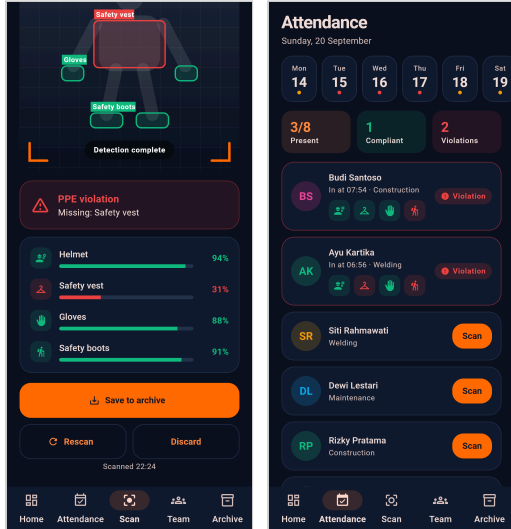


Fig. 1. Scan result (left) and attendance view (right).

D. Persistence and reporting

Employees and scans are stored as versioned JSON in platform key–value storage (mobile, desktop and web). A generator produces a branded A4 PDF with summary indicators and a per-scan table for one scan, one worker or the current archive filter.

V. IMPLEMENTATION AND VERIFICATION

The implementation targets Flutter 3.41 / Dart 3.11 with 32 source files ($\approx 4.9k$ lines) and Android, iOS, web, Windows and macOS runners. It passes `flutter analyze` with no errors or warnings under the `very_good_analysis` rule set (two informational lints remain) and is covered by 38 automated tests (Section VI-F).

The current build uses a simulated detector that draws item confidences at random after a short delay. It exists to exercise the whole workflow without a camera or trained model; consequently *no detection accuracy is claimed in this paper*. The authors' earlier prototype (`ppe_apk_prototype`) ships a YOLO model; connecting it behind `PpeDetector` is the immediate next step.

E. Author's earlier prototype (measured facts)

The author's earlier Android prototype (`ppe_apk_prototype`) uses a YOLOv8-format detector exported to TFLite/ONNX (the TFLite file is 6.2 MB, distributed via GitHub Releases) with six classes: Gloves, Vest, goggles, helmet, mask and safety_shoe. Inference runs natively (Kotlin, Ultralytics YOLO library) with input size 640×640 , confidence threshold 0.25 and NMS IoU threshold 0.45; records are stored in SQLite and reports exported as PDF. The repository includes 11 validation photographs (helmets, vests, safety shoes). GearGuard maps *safety_shoe* to its *boots* item; the other five classes map one-to-one.

VI. EVALUATION

A. Preliminary detector smoke test (measured)

We ran the author's TFLite model (6.2 MB; input $1 \times 640 \times 640 \times 3$, output $1 \times 10 \times 8400$) on the 11 validation photographs in the prototype repository: five helmet, three vest and three safety-shoe images, one object each. Labels are *weak*: the expected class is taken from the file name, and there are no ground-truth boxes, so mAP cannot be computed. With the app's settings (confidence 0.25, class-wise NMS IoU 0.45) the expected class was detected in 11/11 images, both with the app's direct 640×640 resize and with standard letterbox preprocessing (Table I). Confidence for safety shoes was markedly lower under direct resize (0.52–0.82) than under letterbox (0.74–0.92), suggesting that aspect-ratio distortion hurts; letterbox is recommended. Mean latency was 79.7 ms per frame (median 79.5, p95 82.1; 30 timed runs after 3 warm-ups; LiteRT, 4 threads) on a desktop AMD Ryzen CPU (Family 25, Model 80) under Windows.

Table I. Expected-class detection on the 11 validation photos.

Class	n	Hit	Mean conf. (app resize)	Mean conf. (letterbox)
helmet	5	5/5	0.89	0.89
Vest	3	3/3	0.79	0.86
safety_shoe	3	3/3	0.64	0.82
All	11	11/11	0.80	0.87

Limits. Eleven positive-only, single-object images: false-positive behaviour is unmeasured; goggles, gloves and mask were not exercised; the photos come from the prototype's own repository and may overlap with its training data (unknown); and desktop-CPU latency is not phone latency. This is a functional check, *not* an accuracy estimate. Script and raw results are in the repository (`tool/eval`, `docs/eval`).

B. Confidence-threshold sensitivity

Table II shows how many of the 11 photos still yield the expected class as the threshold τ rises. Letterbox

preprocessing keeps 11/11 up to $\tau = 0.7$ and 10/11 at 0.8, whereas the app's direct resize drops to 9/11 at $\tau = 0.6$. At $\tau = 0.9$ recall collapses (5/11 and 3/11). A site that wants strict rules must therefore weigh missed equipment against false alarms; our default of 0.6 keeps 11/11 with letterbox.

Table II. Photos (of 11) with the expected class vs. τ .

τ	0.25	0.40	0.50	0.60	0.70	0.80	0.90
app resize	11	11	11	9	9	8	3
letterbox	11	11	11	11	11	10	5

C. Robustness to perturbations

Each photo was perturbed (seeded; letterbox, $\tau = 0.25$) and re-scored (Table III). The detector is insensitive to darkening, JPEG compression, $\pm 15^\circ$ rotation, sensor noise and $4\times$ down-scaling (11/11). Strong over-exposure (10/11), heavy blur (8/11 at radius 8) and especially a random grey patch covering 25% of the image (5/11, mean confidence 0.30) hurt. Occlusion is the most relevant real-site failure mode (bags, other workers, equipment) and is not modelled by the single-frame rule of Section III.

Table III. Effect of perturbations ($n = 11$).

Perturbation	Hits/11	Mean conf.	Wrong-class boxes
original	11	0.87	0
dark x0.4	11	0.87	0
bright x1.8	10	0.77	0
blur r=3	11	0.85	1
blur r=8	8	0.59	1
JPEG q=20	11	0.87	0
noise sigma=0.08	11	0.88	1
rotate +15 deg	11	0.87	0
rotate -15 deg	11	0.85	0
occlusion 25%	5	0.30	1
downscale x0.25	11	0.87	0

D. Negative images

On 15 non-PPE images (two natural photographs, six solid colours, six noise images, one gradient) the model produced 2 detections at $\tau = 0.25$ and 1 at $\tau = 0.5$ and 0.7. Both false positives were *Vest* on a plain colour patch: orange-red (RGB 212, 57, 14) at confidence 0.81 and pink (240, 159, 174) at 0.39. The natural photographs produced none. This points to a colour bias toward high-visibility hues, so thresholds alone cannot remove the orange case, and hard-negative training data or colour augmentation is advisable. The set is small and synthetic; it indicates a failure mode rather than a false-positive rate.

E. Latency

Latency scales with CPU threads (Table IV), from 242.6 ms on one thread to 70.2 ms on eight (50 timed runs after 3 warm-ups, desktop CPU). Phone latency remains unmeasured.

Table IV. Inference latency vs. threads.

Threads	Mean (ms)	p95 (ms)
1	242.6	247.3
2	137.5	141.0
4	87.0	89.0
8	70.2	72.6

F. Software verification

The suite now has 38 automated tests (unit, state, PDF generation, persistence and end-to-end widget tests) with **75.1%** line coverage (1,532 of 2,040 lines; domain 100%, data 95%, core 92%, state 80%, feature screens 67%). Twenty accessibility checks (Flutter's tap-target, labelled-target and WCAG text-contrast guidelines on five screens $\times$ light/dark) pass. These checks initially *failed* on three light-theme screens, with contrast as low as 2.13:1: orange text on tinted orange surfaces. The fix darkened the light-theme primary from #E85D00 to #A83600 (6.1:1 on the surface) and derives status-text shades algorithmically, after which all checks pass. A release web build has a 3.2 MB main script.

G. Usability study (not conducted)

No user study has been run, so no usability claim is made. A protocol, a bilingual System Usability Scale questionnaire and a scoring script are provided in the repository (docs/paper/study); results should be interpreted against the benchmark mean of 68 (SD 12.5) [6]. The automated accessibility checks above are not a substitute for testing with supervisors.

VII. LIMITATIONS

Compliance is decided per item from a single frame; occlusion, lighting and multiple workers per frame are not modelled. Identity is selected from a list rather than recognised. Storage is local to the device. Camera-based monitoring of people raises privacy and consent issues; deployments must define retention and consent policies and should store scores rather than images. In Indonesia, processing images of identifiable workers falls under the Personal Data Protection Law [7], which requires a lawful basis and purpose limitation.

VIII. CONCLUSION

We presented GearGuard, a detector-agnostic architecture and open-source implementation of the PPE compliance workflow. Future work: a real on-device detector behind the same interface, a field usability study and department-level reporting.

AUTHOR STATEMENT

Single-author work by Ahmad Fauzan Prayogi (conception, software, analysis, writing). Generative AI tools were used to assist in drafting the software and the manuscript; the author reviewed all content and is responsible for it.

REFERENCES

1. Z. Wang, Y. Wu, L. Yang, A. Thirunavukarasu, C. Evison, Y. Zhao, "Fast Personal Protective Equipment Detection for Real

- Construction Sites Using Deep Learning Approaches,” *Sensors*, vol. 21, no. 10, Art. 3478, 2021, doi: 10.3390/s21103478.
2. J. Karlsson, F. Strand, J. Bigun, F. Alonso-Fernandez, K. Hernandez-Diaz, F. Nilsson, “Visual Detection of Personal Protective Equipment and Safety Gear on Industry Workers,” arXiv:2212.04794, 2022.
3. C.-Y. Wang, A. Bochkovskiy, H.-Y. M. Liao, “YOLOv7: Trainable Bag-of-Freebies Sets New State-of-the-Art for Real-Time Object Detectors,” in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2023, pp. 7464–7475.
4. G. Jocher, A. Chaurasia, J. Qiu, “Ultralytics YOLOv8,” software, v8.0.0, 2023. [Online]. Available: <https://github.com/ultralytics/ultralytics>
5. J. Ordick, G. H. Wibowo, A. Fahmi, I. Kurniawan, E. S. Haq, “YOLOv11-Based Automated PPE Detection System for Workplace Safety Monitoring in Electric Power Distribution Operations,” *Journal of Information Systems and Informatics*, vol. 7, no. 4, pp. 4294–4329, 2025, doi: 10.63158/journalisi.v7i4.1379.
6. M. Hyzy, R. Bond, M. Mulvenna, L. Bai, A. Dix, S. Leigh, S. Hunt, “System Usability Scale Benchmarking for Digital Health Apps: Meta-analysis,” *JMIR mHealth and uHealth*, vol. 10, no. 8, e37290, 2022, doi: 10.2196/37290.
7. Republic of Indonesia, “Law No. 27 of 2022 on Personal Data Protection (Undang-Undang Nomor 27 Tahun 2022 tentang Pelindungan Data Pribadi),” 2022.

Verify every reference against the publisher page before submission.